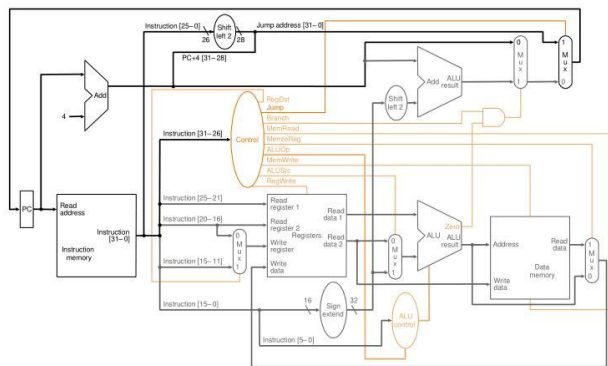


The single cycle CPU



1

Copyright 1998 Morgan Kaufmann Publishers, Inc. All rights reserved.

MIDTERM1 PROJECT

Computer Architecture

A “short” document to explain the project.

Zeinab Azordeh

Student code: 40317653

The ALU Implementation:

ALU is the simplest part of this project, which I wrote first.

```
module Alu (
    input wire[2:0] operation,
    input wire[15:0] in1, in2,
    output wire[15:0] result,
    output wire zero_flag,
    output wire overflow_flag
);

wire[15:0] alu_result;

assign alu_result = (operation == 3b'000) ? (in1 + in2) :
    (operation == 3b'001) ? (in1 - in2) :
    (operation == 3b'010) ? (in1 & in2) :
    (operation == 3b'011) ? (in1 | in2) :
    (operation == 3b'100) ? (not in1) :
    (operation == 3b'101) ? (in1) :
    (operation == 3b'110) ? (in2) :
    16'h0000;
assign result = alu_result;
assign zero_flag = (alu_result == 16'h0000)
```

All defined operations and flags:

Then the overflow flag was a little tricky, in the add operation if sign bits are the same but the result's sign bit isn't same as the operands; we have overflow, and in the sub operation if the sign bits differ:

$X - (-Y) = X + Y$, which shows eventually the result must have the same sign bit as the first input or in1.

```
// Overflow in ADD: signs same, result sign different
// Overflow in SUB: signs different, result sign different from first operand
wire add_overflow = (in1[15] == in2[15]) && (in1[15] != alu_result[15]) && (operation == 3'b000);
wire sub_overflow = (in1[15] != in2[15]) && (in1[15] != alu_result[15]) && (operation == 3'b001);

assign overflow_flag = add_overflow | sub_overflow;

endmodule
```

REGISTER MODULE:

This module stores the logic of registers; we must have eight 16-bit registers.

Everything works on the clock. Each clock we have either a reset signal or a write enable. We only handle writing in registers here. This module also has two 3_bit address input (rd_addr1 and rd_addr2) and one 3_bit write address input. One 16_bit data to write input and two 16_bit data to read to show our computer the data we have.

```
timescale 1ns / 1ps

module registers (
    input      clk,           // System clock
    input      reset,         // Asynchronous reset (active high)
    input      we,            // Write enable
    input [2:0] rd_addr1,     // Read address 1
    input [2:0] rd_addr2,     // Read address 2
    input [2:0] wr_addr,      // Write address
    input [15:0] wr_data,     // Data to write

    output [15:0] rd_data1,   // Data from read address 1
    output [15:0] rd_data2   // Data from read address 2
);
```

Then before working the clock out, we must take care of assigning each register to their own address:

```
// 8 registers, each 16 bits wide
reg [15:0] reg_file [0:7];

assign rd_data1 = reg_file[rd_addr1];
assign rd_data2 = reg_file[rd_addr2];
```

This always block works on each clock to handle the program correctly.

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        // Clear all registers on reset
        reg_file[0] <= 16'h0000;
        reg_file[1] <= 16'h0000;
        reg_file[2] <= 16'h0000;
        reg_file[3] <= 16'h0000;
        reg_file[4] <= 16'h0000;
        reg_file[5] <= 16'h0000;
        reg_file[6] <= 16'h0000;
        reg_file[7] <= 16'h0000;
    end else if (we) begin
        // Write to register on clock edge if write enabled
        reg_file[wr_addr] <= wr_data;
    end
end

endmodule
```

THE MEMORY: 4K × 16

Another simple code we have here, just an array implementation with reading and writing abilities. After reset or in the beginning we have all zeros in our memory. Then if the write enable is active, we write in the address given to us, if the read signal is active, we read from the given address.

```
`timescale 1ns / 1ps

module memory (
    input      clk,           // System clock
    input      reset,         // Asynchronous reset

    // Port 1: Instruction Fetch (Read-Only)
    input  [11:0] inst_addr,   // Address from Program Counter (PC)
    output [15:0] inst_out,    // 16-bit Instruction sent to Control Unit

    // Port 2: Data Access (Read & Write)
    input      data_we,        // Data write enable (from Control Unit)
    input  [11:0] data_addr,    // Address from Instruction (adr-12)
    input  [15:0] data_in,      // Data to write (from Accumulator R0)
    output [15:0] data_out      // Data read (to Register File)
);

    // Memory array
    reg [15:0] mem [0:4095];

    // Asynchronous Read Ports
    assign inst_out = mem[inst_addr];
    assign data_out = mem[data_addr];
```

In one always block we handle reset and write:

```
// Synchronous Write Port
integer i;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        // Clear entire memory on reset
        for (i = 0; i < 4096; i = i + 1) begin
            mem[i] <= 16'h0000;
        end
    end else if (data_we) begin
        // Write data to memory on clock edge if write enabled
        mem[data_addr] <= data_in;
    end
end
```

The reset behavior is similar to an initial begin block. If needed, we can change it so that the registers are initialized from a file.

CONTROL UNIT:

We need a module to set the right signals for each instruction. Actually, on easy terms: we need a boss to tell each of his employees when to start their work and how to do it.

On more complex terms: *The control unit decodes the current instruction and generates all control signals, register addresses, ALU operation codes, memory control signals, and program-counter selection signals required for instruction execution. It does not perform arithmetic operations or store data itself.*

```
timescale 1ns / 1ps

module control (
    input  [15:0] instruction,      // 16-bit instruction
    input  zero_flag,              // Zero flag from ALU
    input  [15:0] pc_current,      // Current program counter

    output reg_write_en,          // Register write enable
    output [2:0] reg_write_addr,  // Register write address
    output [1:0] reg_src_sel,     // Register source select
    output [2:0] alu_op,          // ALU operation
    output mem_read_en,           // Memory read enable
    output mem_write_en,         // Memory write enable
    output [1:0] pc_next_sel,     // Next PC select
    output [11:0] branch_addr,    // Branch/Jump address
    output [2:0] reg_read_addr1,  // Primary register read address
    output [2:0] reg_read_addr2  // Secondary register read address
);
```

All input and output help with how the boss (control unit) controls its employees (registers, ALU, pc, etc.).

- Each instruction has 16_bit and program counter points to an address therefore must be 16_bit itself.
- Then controller must control registers so outputs the write enable and write address and register source select (It acts as a switch selecting between: The result coming out of the ALU, the data coming from Memory, A hardcoded constant/immediate value from the instruction itself). Registers have read addresses too (2_3_bit outputs)
- For ALU we only need to know the operation.
- Also, the boss handles memory read and write switches.
- If the processor is on a branch or jump or else not, the pc next select differs, therefore we need to determine this as well (keeping the branch or jump address).

```
wire [3:0] opcode   = instruction[15:12];
wire [2:0] reg_index = instruction[11:9];
wire [11:0] addr_12  = instruction[11:0];
wire [8:0] addr_9    = instruction[8:0];    // Extracted for BranchZ
wire [8:0] func      = instruction[8:0];

// Constant Assignments
assign reg_read_addr1 = 3'b000;           // R0 is the explicit Accumulator
assign reg_read_addr2 = reg_index;        // Always output Ri to the second read port
```

The left most 4_bits are the opcode, and the rest of the bits show different values according to each operation. First register to read is always linked to R0, also known as accumulator.

```
// Combinatorial instruction decode
always @(*) begin
    // Default safe values (prevents latch generation)
    reg_write_en = 1'b0;
    reg_write_addr = 3'b000;
    reg_src_sel = 2'b00;
    alu_src_sel = 1'b0;
    alu_op = 3'b101;
    mem_read_en = 1'b0;
    mem_write_en = 1'b0;
    pc_next_sel = 2'b00;
    branch_addr = addr_12; // Default target is the 12-bit address

case (opcode)
```

- Type A (Opcode 0000-0010): Load, Store, Jump

1. LOAD

```
// Type A Instructions
4'b0000: begin // LOAD adr-12
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    reg_src_sel = 2'b01;
    mem_read_en = 1'b1;
    alu_src_sel = 1'b1;
end
```

2. STORE

```
4'b0001: begin // STORE adr-12
    mem_write_en = 1'b1;
    alu_src_sel = 1'b1;
end
```

3. JUMP

```
4'b0010: begin // JUMP adr-12
    pc_next_sel = 2'b01;
    branch_addr = addr_12; // PC <- adr-12
end
```

- Type B (Opcode 0100): Branch

1. BRANCHZ

```
// Type B Instructions
4'b0100: begin // BRANCHZ Ri, adr-9
    alu_op = 3'b001; // SUB to evaluate zero_flag
    alu_src_sel = 1'b0;

    // PC[8:0] <- adr-9. Combine top 3 bits of PC with 9-bit instruction address.
    branch_addr = {pc_current[11:9], addr_9};

    if (zero_flag) begin
        pc_next_sel = 2'b10;
    end else begin
        pc_next_sel = 2'b00;
    end
end
```

The if -> jumping logic

The else -> normal PC logic

- Type C (Opcode 1000): Register operations

1. REGISTER OPS

```
// Type C Instructions (Register Operations)
4'b1000: begin
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    reg_src_sel = 2'b00;
    alu_src_sel = 1'b0;

    case (func)
        9'b000000001: begin // MOVETO Ri
            reg_write_addr = reg_index;
            alu_op = 3'b101;
        end
        9'b000000010: alu_op = 3'b110; // MOVEFROM Ri
        9'b000000100: alu_op = 3'b000; // ADD
        9'b000001000: alu_op = 3'b001; // SUB
        9'b000010000: alu_op = 3'b010; // AND
        9'b000100000: alu_op = 3'b011; // OR
        9'b001000000: alu_op = 3'b100; // NOT
        9'b010000000: begin // NOP
            reg_write_en = 1'b0;
        end
        default: reg_write_en = 1'b0;
    endcase
end
```

- Type D (Opcode 1100-1111): Immediate operations

```
4'b1100: begin // ADDI
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    alu_src_sel = 1'b1;
    alu_op = 3'b000;
end

4'b1101: begin // SUBI
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    alu_src_sel = 1'b1;
    alu_op = 3'b001;
end

4'b1110: begin // ANDI
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    alu_src_sel = 1'b1;
    alu_op = 3'b010;
end

4'b1111: begin // ORI
    reg_write_en = 1'b1;
    reg_write_addr = 3'b000;
    alu_src_sel = 1'b1;
    alu_op = 3'b011;
end

default: begin
    reg_write_en = 1'b0;
end
endcase
end
```

In all 4 immediate operations, reg write is active, and the address to write inside is R0. Then ALU second operand select is 1 which is immediate. And each operation is chosen according to its code.

PROCESSOR:

The processor module is a glue to everything we coded so far.

```
`timescale 1ns / 1ps

module processor (
    input      clk,           // System clock
    input      reset          // Asynchronous reset (active high)
);
```

Only inputs are clock and reset, and no outputs. This module is like the whole company which has all the inputs and outputs we have defined in the last 5 modules as inside regs and wires:

```
// Program Counter
reg [15:0] pc;
wire [15:0] pc_next;

// Instruction
wire [15:0] instruction;

// Register file signals
wire      reg_write_en;
wire [2:0] reg_write_addr;
wire [15:0] reg_write_data;
wire [2:0] reg_read_addr1;
wire [2:0] reg_read_addr2;
wire [15:0] reg_data1;
wire [15:0] reg_data2;

// ALU signals
wire [2:0] alu_op;
wire [15:0] alu_in1;
wire [15:0] alu_in2;
wire [15:0] alu_result;
wire      zero_flag;
wire      overflow_flag;

// Memory signals
wire      mem_read_en;
wire      mem_write_en;
wire [11:0] mem_addr;
wire [15:0] mem_data_out;

// Control signals
wire [1:0] reg_src_sel;
wire [1:0] pc_next_sel;
wire [11:0] branch_addr;
wire      alu_src_sel;
```

```
// Instruction Memory ROM
reg [15:0] instr_mem [0:4095]; // 4K instructions

initial begin
    // Initialize instruction memory with zeros
    integer i;
    for (i = 0; i < 4096; i = i + 1) begin
        instr_mem[i] = 16'h0000;
    end
end

// Fetch instruction from program memory
assign instruction = instr_mem[pc[11:0]];
```

Here we initialize our memory to 4K of 16_bit zeros and fetch the instruction using 12 lines of address.

```
assign pc_next = (pc_next_sel == 2'b00) ? (pc + 1) :
                 (pc_next_sel == 2'b01) ? {4'b0000, branch_addr} :
                 (pc_next_sel == 2'b10) ? {4'b0000, branch_addr} :
                 (pc + 1);

// Update PC on clock edge
always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc <= 16'h0000;
    end else begin
        pc <= pc_next;
    end
end
```

We decide what next program counter is going to point and sync it with the clock.

After declaring all registers and wires, initializing memory, refining next pc and its logic, we will assign each wire or register to its own module. Control, Register, Memory and ALU. But before defining ALU we need to declare alu_in1 (always accumulator) and alu_in2 (sometimes a register, sometimes an immediate):

```
assign alu_in1 = reg_data1; // Always R0 for Type A, B, C, D instructions

//Using the control signal to switch between Immediate or Register
assign alu_in2 = (alu_src_sel) ? {4'h0, instruction[11:0]} : reg_data2;

Alu alu_unit (
    .operation(alu_op),
    .in1(alu_in1),
    .in2(alu_in2),

    .result(alu_result),
    .zero_flag(zero_flag),
    .overflow_flag(overflow_flag)
);
```

At the end we form the output, in a simple way we just need to tell our output port, where to get its information from:

```
assign reg_write_data = (reg_src_sel == 2'b00) ? alu_result :
                        (reg_src_sel == 2'b01) ? mem_data_out :
                        (reg_src_sel == 2'b10) ? pc :
                        alu_result;
```

And it all works out perfectly. :)